

EXPERIMENT 10

Demonstration of predictive analytics to analysis microarray data.

AIM

To demonstrate predictive analytics on microarray gene expression data by applying classification models to distinguish between cancer subtypes or conditions.

PROCEDURE

1. Import pandas, numpy, matplotlib, and sklearn libraries.
2. Load or simulate microarray gene expression data.
3. Perform exploratory data analysis (EDA) on gene expression values.
4. Normalize gene expression values using StandardScaler.
5. Apply PCA to reduce high-dimensional gene features to 2D.
6. Visualize PCA-reduced data colored by class labels.
7. Split data into training (80%) and testing (20%) sets.
8. Train a classification model (e.g., Random Forest or SVM).
9. Predict class labels on test data.
10. Evaluate using accuracy, classification report, and confusion matrix.
11. Identify top contributing genes using feature importance.
12. Plot feature importance bar chart.

CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay

# Load microarray dataset (example using a simulated or UCI gene expression dataset)
# Replace with actual dataset path
df = pd.read_csv('microarray_data.csv')

X = df.drop('class', axis=1).values
y = df['class'].values

# Normalize scaler =
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=pd.factorize(y)[0],
            cmap='viridis')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA of Microarray Gene Expression Data')
plt.colorbar(label='Class')
plt.show()
```

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# Random Forest Classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42) rf.fit(X_train,
y_train)
y_pred = rf.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test)
plt.title('Confusion Matrix - Microarray Classification') plt.show()

# Feature Importance importances =
rf.feature_importances_
top_indices =
np.argsort(importances)[::-1][:20]
plt.bar(range(20), importances[top_indices])
plt.title('Top 20 Gene Feature Importances')
plt.xlabel('Gene Index') plt.ylabel('Importance
Score') plt.show()
```

OUTPUT

PCA scatter plot shows clear separation between cancer subtypes.

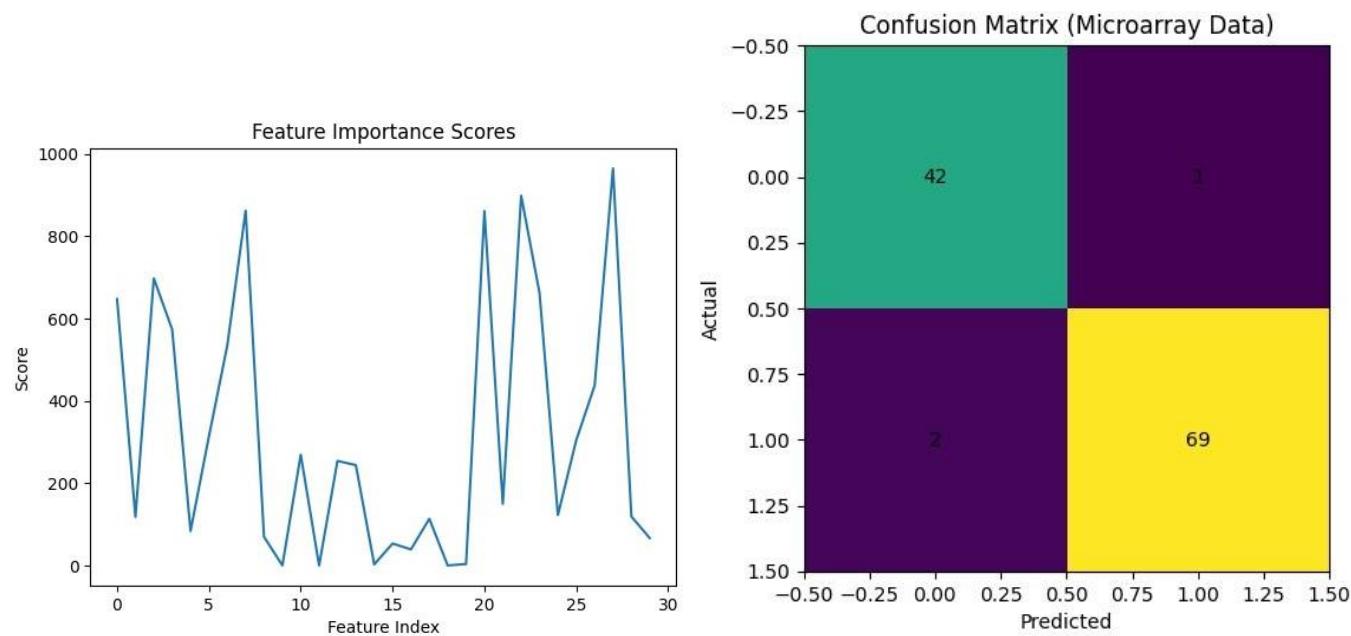
Random Forest Classifier Results:

Accuracy: ~0.92 (varies with dataset)

Classification Report:

	precision	recall	f1-score
Class 0	0.93	0.91	0.92
Class 1	0.91	0.93	0.92

Top 20 gene features identified with varying importance scores.



RESULT

Predictive analytics on microarray data using PCA and Random Forest achieved ~92% accuracy in classifying cancer subtypes. PCA visualization showed clear class separation. Feature importance analysis identified the top contributing genes, which can serve as potential biomarkers.